



USC University of
Southern California

Working inside Software 3.0

CSCI 599 — Agentic AI

Week 1, Lecture 2

USC



Where we left off

Last time: what Software 3.0 looks like in a real repo

- OpenClaw — workspace files as English, skills as Markdown specs, Docker as the 1.0 floor
- The three layers, side by side, in one project

Today: what your job looks like inside that stack

- Same three layers — now you're the one authoring at the top

Today's arc

Three concepts, one demo, two opinions

- IDE + AI assistants — what tools you actually use
- Repository instructions — AGENTS.md as the standard
- From vibe coding to agentic engineering — discipline beyond the slogan

Mid-lecture: live build of a tiny agent in Codex

IDE + AI Assistants



The IDE became an agent host

Editors evolved into places where agents read, write, and run code

- 2022 — Copilot: autocomplete the next few tokens
- 2023 — Cursor: chat and editing with codebase context
- 2026 — coding agents operate across apps, CLIs, and IDE integrations

Instructions join code as a first-class artifact

Three modes, in order of trust required

Autocomplete — the agent suggests, you accept token-by-token

- Failure mode: confident plausible code that's subtly wrong

Chat-with-context — sidebar agent, reads the repo, you ask, it answers

- Failure mode: stale context, hallucinated file paths

Agentic — the agent edits files and runs commands on its own

- Failure mode: cascading wrong moves before you notice

Codex (yours) and Claude Code (the peer)

	OpenAI Codex	Claude Code
Model	GPT-5.6 Sol is the recommended default	Configurable Claude model (Opus or Sonnet)
Surfaces	Desktop, web, CLI, cloud, iOS, and IDE integrations	CLI, IDE integrations, web, and Slack
Repo instructions	AGENTS.md (concatenated into context at session start)	CLAUDE.md (Anthropic-specific) + AGENTS.md (fallback in many tools)
Course access	USC ChatGPT Edu OAuth; included, with 5-hour and weekly limits	Separately funded Anthropic Console API key

The permission model is the safety surface

Current Codex permission profiles (`/permissions`` to switch):

- ``:read-only`` — inspect and plan, no writes
- ``:workspace`` — writes only in workspace roots and system temp
- ``:danger-full-access`` — no local sandbox restrictions

Sandbox boundaries and approval policy are separate controls

- Legacy `--full-auto`` is deprecated; prefer workspace permissions

Use least privilege, then approve only the exceptions you understand

Why the IDE matters at all

These tools are useful because the codebase IS the context

- The prompt isn't an island — it's a spec evaluated against a real repo
- Which is why per-repo instructions matter (next section)

The 2026 landscape, two data points:

- NVIDIA released an early NemoClaw preview at GTC 2026 (March 16)
- OpenClaw's creator joined OpenAI in Feb 2026; project moved to an independent foundation

Telling Agents How Your Repo Works



The problem

Without a per-repo instructions file, every session restarts cold

- Agent suggests `npm test` when the repo uses `yarn`
- Agent invents a build command instead of reading package.json
- Agent edits a generated file you wished it would leave alone

Each fix is a prompt that should have been written once, in the repo

AGENTS.md, in one slide

Plain Markdown at the root of the repo. No required schema.

- Released August 2025; 60,000+ repos adopt it (per OpenAI, Dec 2025)
- Released by OpenAI; adopted by Codex, Amp, Jules, Cursor, Factory, and others
- Donated to AAIF Dec 9, 2025 alongside MCP and goose

Codex behavior: AGENTS.md is concatenated into context at session start

- Monorepos: root-to-directory chain; closer instructions override conflicts

The fragmentation AGENTS.md replaced

File	Tool	Status
.cursor/rules	Cursor	Current native rules; .cursorrules is legacy
.windsurf/rules	Windsurf	Current native rules; older single-file rules were predecessors
AGENTS.md	Many major coding agents	Shared plain-Markdown convention

NOTE

Vendor-specific rule systems still matter, but AGENTS.md provides one portable instruction file across many major coding agents.

CLAUDE.md alongside

Anthropic-specific, predates AGENTS.md, still in use

- Provides Claude-specific project instructions and persistent context
- Permissions and hooks use settings; MCP servers use ``.mcp.json``

You'll see CLAUDE.md in real repos; you won't write one for assignments

The minimum viable AGENTS.md

What every assignment AGENTS.md should contain:

- How to install dependencies — exact command
- How to run the project — exact command
- How to run the tests — exact command
- Code style — Python version, formatter, lint rules
- What NOT to touch — generated files, secrets, vendored code

```
# AGENTS.md
## Setup
pip install -r requirements.txt
## Run
python -m assistant
## Test
pytest tests/
## Style
Python 3.11+, ruff for lint, black for format.
## Do not touch
- .env - generated/*.py - vendor/
```

Papa's rule

PAPA'S RULE

Write AGENTS.md before you start vibe-coding, not after the agent has gone off the rails three times.

The exception:

- First five minutes of a throwaway prototype — fine, skip it
- The moment it's worth committing to git, it's worth an AGENTS.md

Treat AGENTS.md as part of the project, not boilerplate

From Vibe Coding to Agentic Engineering



What vibe coding actually is

"There's a new kind of coding I call 'vibe coding,' where you fully give in to the vibes, embrace exponentials, and forget that the code even exists."

"I see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works."

— Andrej Karpathy, X, February 2, 2025. Karpathy's same post notes the approach suits non-critical projects; the code grows beyond his understanding.

2026: vibe coding became agentic engineering

	Vibe coding (2025)	Agentic engineering (2026)
Best fit	Low-stakes experiments and prototypes	Professional software and durable systems
Human role	Describe, observe, keep moving	Specify, supervise, test, and own the result
Quality bar	Output can outrun understanding	Claim the leverage without compromising quality

COURSE METHOD

This course uses agentic engineering: AGENTS.md + bounded permissions + verification + review.

The agentic engineering loop

Four steps, in order:

- Describe — state the intent precisely (this is where AGENTS.md pays off)
- Run — let the agent produce; let it run its code
- Verify — check the behavior; you don't have to read every line, but you do have to confirm it works
- Refine — feedback the failure or the gap, then loop

Vibe coding vs agentic engineering

Dimension	Vibe coding	Agentic engineering
Intent	Explore quickly; constraints may stay implicit	Specify behavior, boundaries, and success criteria
Oversight	Accept momentum and tolerate uncertainty	Inspect evidence, tests, and surprising diffs
Failure	Re-prompt until the experiment works	Diagnose, refine constraints, or take over
Ownership	Appropriate for throwaway or low-stakes work	Human remains accountable for the result

Three failure modes, three recoveries

Confidently wrong code

- Agent produces working-looking output that's subtly broken
- Recovery: behavioral verification — run it, test the edges

Stuck loops

- Agent tries the same fix repeatedly, can't escape its frame
- Recovery: escalate — restart with different framing, or write it yourself

Scope creep

- Agent edits files far from the task, leaves a wake of changes
- Recovery: AGENTS.md scoping + permission mode + smaller prompts

Live: Agentic Engineering with a Tiny CLI



Live: building a weather CLI in Codex

LIVE DEMO

Build — a Python CLI that calls wtr.in for a 3-day forecast

Authenticate — USC ChatGPT Edu OAuth; no OpenAI API key

Constrain — write AGENTS.md first and use workspace-restricted permissions

Watch — how instructions shape the first actions and where verification catches a gap

Test — request a save to ~/Desktop; distinguish instruction adherence from sandbox enforcement

Network — wtr.in access may require approval

Review — recorded version on Brightspace

What we just saw

AGENTS.md shaped the agent's first actions

Correction happened in the verify step, not the prompt step

AGENTS.md shaped the response; the sandbox supplied the enforceable boundary

What This Means for the Course



The assignments use agentic engineering with verification

Every assignment assumes Codex + AGENTS.md + verification as the method

- Assignment 1 — Tool-using agent with MCP integration
- Assignment 2 — Multi-agent system with RAG
- Assignment 3 — OpenClaw Personal Assistant
- Assignment 4 — Capstone, original agentic application

The Process Log (10% of each assignment) is the verify-step receipt

Install before next class

Codex — the app, the CLI, or the IDE extension (your choice)

- Sign in with USC ChatGPT Edu OAuth — no OpenAI API key
- Verify the install runs: ``codex --version`` in a fresh terminal
- Included is not unlimited: 5-hour windows + weekly limits; check ``/status``

A template AGENTS.md for Assignment 1 will be on Brightspace

Bring questions to office hours — install issues are the boring kind, let's get them out of the way

Papa's rule

PAPA'S RULE

The slowest part of agentic engineering is verification. If you skip it, you're not faster — you're shipping someone else's bugs.

What "verify" actually means in practice:

- Run it. Test the edges. Read the diff for what surprised you.
- Not: read every line. Not: skip it because the output looked fine.

The Process Log is your verification trail — write it as you go

Lecture 1 + Lecture 2, together

Layer	In OpenClaw (Lecture 1)	In your work (Lecture 2)
What you author	SOUL.md, USER.md, SKILL.md	AGENTS.md, prompts, the spec
What reasons	The LLM the agent calls	GPT-5.6 family (Sol default)
What runs it	TypeScript runtime in Docker	Codex permission profile + sandbox

Where we are, end of Week 1

You have the framework — Karpathy's 1.0 → 2.0 → 3.0

You have the agentic engineering method — describe, run, verify, refine

You have the tools — Codex, AGENTS.md, the permission model

Questions

Two openers if you want one:

- What did the demo not show that you wanted to see?
- What's the part of agentic engineering you're most skeptical of?



Next time: LLMs as agents

Week 2 — the model layer you'll use in Assignment 1

- Cloud models — Gemini, GPT, Claude — as agent backends
- Local models — Gemma 4 via Ollama — when and why
- Prompting techniques and configurable thinking